

第6章 面向对象与 trait

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:熟悉 C 的结构体与函数指针,了解"多态"的概念(不要求写过 OOP)

🎯 本章学习目标

- 回顾 C 用"结构体 + 函数指针表"模拟多态的两种做法,理解其样板代码与类型安全之痛
- 掌握 **trait** 的本质:一组行为契约——定义、为类型实现、默认方法、调用方式
- 理解"无继承"的设计:数据与行为分离,接口通过 trait 表达,复用通过组合实现
- 掌握静态分派:泛型 + trait bound、impl Trait 参数与返回值(对应 C++ 模板,零虚表开销)
- 掌握动态分派: **dyn Trait** 与 **vtable** 的内存布局,与 C 函数指针表、C++ 虚函数对照
- 掌握组合优于继承:has-a 结构体嵌套,替代 C++/Java 的 is-a 继承链
- 会用 `std::ops` 实现运算符重载,理解 `Display/Debug/PartialEq` 等标准 trait 的用法
- 了解 trait 继承与默认方法:接口层面的"继承",用扩展与组合而不是类继承

💡 从 C 到 Rust:本章主题如何迁移

C 程序员对"多态"并不陌生——只是名字不叫多态。常见做法有两种:**tag 分发**(struct 里放一个 kind 字段,switch 分发)和**函数指针表**(struct 里放一个函数指针数组,把方法挂上去)。前者每次加新类型要改 switch,后者每次加新方法要改所有表。两个方案都靠约定维护:编译器不知道"面积函数"该有几个参数、哪个字段配哪个函数。C 的 `void*` 更是把类型检查整个扔掉了。

Rust 的答案叫 **trait**:先声明"会算什么(方法签名)",再让具体类型声明"我实现了它"。编译器全程检查:调用方拿到的是实现了某 trait 的类型,字段与方法——对应,`void*` 消失。这解决了 C 样板代码与类型安全的问题,却保留了两样 C 程序员珍惜的东西:**零开销与确定性**。没有 GC、没有反射、没有运行时类型信息包袱——trait 要么在编译期展开(静态分派,像 C++ 模板),要么放进一张显式的函数指针表(动态分派,像 C 的 `vtbl`,只是由编译器生成)。

另一个关键决裂是"无继承"。C 本身没有继承,所以这对你不是损失;但如果你写过一点 C++/Java,请把"is-a"思维换成"has-a":`Car` 里放一个 `Engine` 字段,而不是 `Engine` 的子类。行为共享用 trait(接口),数据复用用组合(字段),两头都不靠"继承基类"这个又耦合又脆弱的机制。本章的路线:先看懂 C 的痛点(\$6.1),再学 trait(\$6.2-\$6.4 由浅入深),然后学组合思维(\$6.5),最后是运算符重载与 trait 家族(\$6.6-\$6.7)。

📦 前置知识自查

- C 的结构体、函数指针、函数指针数组;typedef 函数指针类型
- C 的 `void*` 与类型强转的后果(类型安全缺失)
- 第4章的 `impl` 方法、`self` 三态(&self / &mut self / self)、关联函数
- 第5章的 `Box` 与所有权——本章的 `Box<dyn Trait>` 依赖它们
- (可选)C++ 的 `virtual` 与虚表概念、Java 的 `interface`,用于对照

本章学习路线

1. **第一阶段(约1.5小时):**§6.1–§6.2, trait 的定义、实现与默认方法——先会"声明契约"
2. **第二阶段(约2小时):**§6.3–§6.4, 静态分派与动态分派——理解"编译期展开"与"运行期查表"的取舍, 这是生产选择的灵魂
3. **第三阶段(约1.5小时):**§6.5–§6.7, 组合思维、运算符重载、标准 trait——把 trait 用进日常代码
4. **验收标准:**给你一个"形状"需求, 能徒手写出 trait + 两种分派 + 组合式扩展, 并解释各自开销

本章知识导图

- §6.1 C 的多态困境: tag 分发与函数指针表
- §6.2 trait: 行为契约的定义、实现与默认方法
- §6.3 静态分派: 泛型 + trait bound、impl Trait
- §6.4 动态分派: dyn Trait、vtable 与对象安全
- §6.5 组合优于继承: has-a 思维替代 is-a
- §6.6 运算符重载与标准 trait: Add / Display / PartialEq
- §6.7 trait 继承与默认方法: 接口层面的扩展
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§6.1 C 的多态困境: tag 分发与函数指针表

6.1.1 方案一: tag 分发

struct 里放一个类型标签, 所有函数开头 switch 一下:

```
// C: tag 分发——"多态"的朴素实现
typedef enum { SHAPE_CIRCLE, SHAPE_RECT } ShapeKind;

typedef struct {
    ShapeKind kind;
    double a;      // 圆: 半径; 矩形: 宽
    double b;      // 矩形: 高(圆时未使用)
} Shape;

double shape_area(const Shape *s) {
    switch (s->kind) {
        case SHAPE_CIRCLE: return 3.14159 * s->a * s->a;
        case SHAPE_RECT:   return s->a * s->b;
    }
    return 0;           // 新类型忘了加分支? 静默返回 0
}
```

痛点一望即知: 加一个新类型(三角形)要改 switch 的每一个地方(面积、周长、打印.....), 漏改一处就静默出错; 字段 a/b 的含义随 kind 变化, 全靠注释约定, 编译器无从检查。

6.1.2 方案二:函数指针表(vtbl)

```
// C:函数指针表—把"方法"挂到结构体上
typedef struct Shape Shape;
typedef struct {
    double (*area)(const Shape *self); // 方法一
    void (*print)(const Shape *self); // 方法二
} ShapeVtbl;

struct Shape {
    const void *data; // 指向具体对象(void*:类型安全已丢失)
    const ShapeVtbl *Vtbl; // 虚函数表指针
};

// 调用:c->Vtbl->area(c) — 每张表都要手工维护,每个函数都要传 self
```

痛点:每加一个方法,所有类型的表都要改;void* 让编译器再也帮不上忙;self 参数靠人肉传递,传错对象编译通过、运行崩溃。C++ 用 virtual 把这张表藏进编译器,但代价是继承体系本身的复杂性。**Rust 的做法:声明契约(trait)由编译器生成表,类型检查全程在线**——这是本章的支点。

C 的多态手段	痛点	Rust 的对应
switch + kind 字段	加类型要改所有分支,漏改静默错	match 穷尽性检查(第3章)让漏改变编译错误
手写函数指针表	表与类型不绑定,void* 丢类型	trait 由编译器生成 vtable,类型全程保留
函数指针参数	签名不携带"来自哪个对象"	fn 指针、闭包(第4章)或 impl Trait

§6.2 trait:行为契约的定义与实现

6.2.1 声明契约

trait 只声明"有哪些行为"(方法签名),不包含数据。数据永远在 struct/enum 里;trait 只描述"实现者保证能做到的事":

```
// 契约:任何实现了 Shape 的类型,必须提供面积与名称
trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> &'static str;
}

// 数据在具体类型里,不在 trait 里
struct Circle { r: f64 }
struct Rect { w: f64, h: f64 }

// 为 Circle 兑现契约
impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.r * self.r
    }
    fn name(&self) -> &'static str {
        "圆形"
    }
}

impl Shape for Rect {
    fn area(&self) -> f64 { self.w * self.h }
    fn name(&self) -> &'static str { "矩形" }
}

fn main() {
    let c = Circle { r: 2.0 };
    println!("{}", c.name(), c.area());
}
```

6.2.2 默认方法:契约带"免费午餐"

trait 里的方法可以带默认实现,实现者只需覆盖需要定制的部分——这是"接口默认方法"(Java 8 同款)的来源,也是后面 trait 继承的原料:

```
trait Greeter {
    fn name(&self) -> String;           // 必须实现

    fn greet(&self) -> String {         // 默认实现:免费赠送
        format!("你好,{}", self.name())
    }
}

struct 小明;
impl Greeter for 小明 {
    fn name(&self) -> String { String::from("小明") }
}

fn main() {
    let x = 小明;
    println!("{}", x.greet());          // 你好,小明!
}
```

⚠ 易错点 1:想把"字段"写进 trait

trait 里只能有方法,不能有字段。常见冲动: `trait Shape { fn area(&self) -> f64; let r: f64; }` ——不行。正确分工: 数据进 struct,行为进 trait;需要"取半径"就声明一个 `fn radius(&self) -> f64` 方法,由各实现者返回自己的字段。这正是 C 的教训:状态与行为分离,表里不放数据。

§6.3 静态分派:泛型 + trait bound

6.3.1 泛型函数:编译期展开,零开销

要求"任意实现了 Shape 的类型"参与计算,用泛型 + trait bound。编译器为每种具体类型生成一份专门的代码(单态化 **monomorphization**,与 C++ 模板一致):

```
// T 被约束:必须实现 Shape。面积之和无需知道具体类型
fn total_area<T: Shape>(shapes: &[T]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

// 等价写法:impl Trait 参数(语法糖,意义相同)
fn total_area2(shapes: &[impl Shape]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

// impl Trait 返回值:隐藏具体类型,只暴露"实现了 Shape"
fn make_circle() -> impl Shape {
    Circle { r: 1.0 }
}

fn main() {
    let shapes = vec![Circle { r: 1.0 }, Circle { r: 2.0 }];
    println!("总面积:{}", total_area(&shapes));

    let c = make_circle();
    println!("返回的隐藏类型:{}", c.name());
}
```

静态分派的特征:类型在编译期确定,没有查表、没有间接跳转,直接内联调用。与 C 的差别:C 里同一份代码操作不同结构体要么靠宏,要么靠 `void*` 丢类型;Rust 里类型信息全程保留,且生成的是"按类型特化"的代码,性能与手写每种类型的专用函数相同。

§6.4 动态分派:dyn Trait 与 vtable

6.4.1 什么时候必须动态分派

集合里放"不同类型的对象"时,泛型做不到——`Vec<Circle>` 只能装圆形。Rust 用 **trait 对象**: `Box<dyn Shape>` (堆上)或 `&dyn Shape` (借用)。运行期通过 vtable 查找方法,与 C 的函数指针表、C++ 的虚函数是同一个机制,只是表由编译器生成、类型安全:

```
fn main() {
    // 异构集合:圆形、矩形装进同一个 Vec
    let shapes: Vec<Box<dyn Shape>> = vec![
        Box::new(Circle { r: 1.0 }),
        Box::new(Rect { w: 2.0, h: 3.0 }),
    ];

    for s in &shapes {
        println!("{}", s.name(), s.area());
    }

    // 借用形式的 trait 对象(不拥有)
    let c = Circle { r: 5.0 };
    let obj: &dyn Shape = &c;
    println!("通过 &dyn 调用:{}", obj.name());
}
```

6.4.2 vtable 的内存真相

★ 重点:trait 对象的内存布局(与 C 的 vtbl 完全同构)

`Box<dyn Shape>` 在内存里是 两个指针(胖指针):① 指向堆上数据的指针;② 指向 vtable 的指针。vtable 是一张表,装着该类型实现的每个方法地址(以及 drop、size 等信息)。调用 `s.area()` 时:先按表地址取 area 的函数指针,再跳转——一次间接调用,与 C 的手写表、C++ 的 virtual 调用开销完全相同。对照:

C 手写 vtbl	C++ virtual	Rust dyn Trait
手工维护表 + void*	编译器生成虚表	编译器生成 vtable
类型安全丢失(void*)	类型安全(静态)	类型安全(静态)
self 参数手动传	this 隐式传	&self 显式传(与普通方法一致)
每类型一份表	每类型一份虚表	每类型一份 vtable
手工增删方法	加方法要改基类	trait 增方法,实现者统一适配
表 + 数据分开存	对象内嵌虚表指针	胖指针:数据指针 + 表指针

6.4.3 对象安全:不是所有 trait 都能 dyn

能放进 dyn 的 trait 必须满足**对象安全**:所有方法都不能有泛型参数(泛型方法无法放进固定大小的表)、不能返回 Self(返回类型因实现而异)、方法不能要求 Sized。违反的典型:

```
// 反例:泛型方法—编译器无法为 dyn 生成 vtable
trait Bad {
    fn compare<T>(&self, other: T) -> bool;    // 泛型方法
}
// let x: Box<dyn Bad>;    // 编译错误:E0038 the trait cannot be made into an object

// 正例:所有方法的具体类型都在签名里固定
trait Good {
    fn compare(&self, other: &dyn Good) -> bool;    // 可以
}

fn main() {
    println!("对象安全:泛型方法无法进 vtable,所以被编译器拒绝");
}
```

⚠ 易错点 2:处处用 dyn,性能与设计双双受损

能静态就不要动态: 泛型/impl Trait 是默认选择,dyn 只在"异构集合""插件化接口""运行期决定类型"时使用。dyn 有两点代价:① 每次调用一次间接跳转(微乎其微,但热路径可感知);② 最关键——失去内联与特化,编译器无法按具体类型优化。生产经验:接口面优先写成泛型;只有当"类型集合在运行期才闭合"(插件、导出表)时才上 dyn。C 程序员换一种说法就懂:能写死类型就别走函数指针。

§6.5 组合优于继承:has-a 思维替代 is-a

6.5.1 继承的问题

C++/Java 的继承给"复用"开了捷径,也埋了雷:基类一旦改变,所有子类的行为跟着变(**脆弱的基类问题**);菱形继承带来重复与歧义;深层继承链让"这个对象到底做什么"越来越难回答。C 没有继承,反而绕开了这些——Rust 延续这个务实传统:数据复用用"包含"(字段),行为复用用"trait",不提供类继承。

6.5.2 组合的标准写法

```
struct Engine {
    power: u32,           // 单位:马力
}

impl Engine {
    fn new(power: u32) -> Engine { Engine { power } }
    fn start(&self) -> String {
        format!("引擎启动,功率 {} 马力", self.power)
    }
}

// 汽车"拥有"引擎:has-a 组合,而不是"是"引擎的子类
struct Car {
    brand: String,
    engine: Engine,        // 关键:嵌套字段
}

impl Car {
    fn new(brand: &str, power: u32) -> Car {
        Car {
            brand: String::from(brand),
            engine: Engine::new(power),
        }
    }

    fn drive(&self) -> String {
        format!("{}", self.brand, self.engine.start())
    }
}

fn main() {
    let car = Car::new("东风", 300);
    println!("{}", car.drive());
}
```

6.5.3 行为复用走 trait,数据复用走组合

需要"Car 也能被驾驶"时,别建"交通工具"基类——声明 trait 让 Car 实现:

```

trait Drivable {
    fn drive(&self) -> String;
}

// Car 实现"可驾驶"契约,同时保留自己的 Engine 字段
impl Drivable for Car {
    fn drive(&self) -> String {
        format!("{}", self.brand, self.engine.start())
    }
}

// 另一个类型同样实现,互不打扰—这是继承做不到的解耦
struct 自行车;
impl Drivable for 自行车 {
    fn drive(&self) -> String { String::from("脚踏驱动,零排放") }
}

fn go_for_a_ride(vehicle: &impl Drivable) {
    println!("出发:{}", vehicle.drive());
}

fn main() {
    go_for_a_ride(&Car::new("东风", 300));
    go_for_a_ride(&自行车);
}

```

C++/Java 继承	问题	Rust 组合方案
class Car : public Vehicle	基类改动波及全部子类	struct Car { engine: Engine } 字段包含
纯虚接口	接口与实现绑定在类层级里	trait 独立于类型声明,可按需实现
菱形继承	重复与歧义	不存在(无继承,无菱形)
向上转型传递依赖	隐式耦合	泛型 + trait bound:显式契约

§6.6 运算符重载与标准 trait 家族

6.6.1 运算符重载:经 std::ops

Rust 的运算符(+, -, ==, 索引等)都是 trait:实现了 `std::ops::Add` 就能用 +。与 C++ 的 operator 重载对应,但每个运算符绑定固定的 trait 与语义,不能发明新符号:

```

use std::ops::Add;

#[derive(Debug, Clone, Copy)]
struct Point {
    x: f64,
    y: f64,
}

// 实现 + :self 是"被加数"(Add 消费两个值)
impl Add for Point {
    type Output = Point;

    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}

fn main() {
    let a = Point { x: 1.0, y: 2.0 };
    let b = Point { x: 3.0, y: 4.0 };
    println!("{:?}", a + b);           // Point { x: 4.0, y: 6.0 }
}

```

6.6.2 显示与比较: Display / Debug / PartialEq

```

use std::fmt;

// Display: {} 格式, 面向用户
impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}

// PartialEq: == 与 !=, 可自定义比较规则(这里用容差)
impl PartialEq for Point {
    fn eq(&self, other: &Self) -> bool {
        (self.x - other.x).abs() < 1e-9
        && (self.y - other.y).abs() < 1e-9
    }
}

fn main() {
    let a = Point { x: 1.0, y: 2.0 };
    let b = Point { x: 1.0, y: 2.0 };
    println!("{a}");                // (1, 2): 走 Display
    println!("{a} == {b} : {}", a == b); // true: 走 PartialEq

    // Debug({:?}) 常用 derive 自动生成, 手写用于定制
    println!("{:?}", a);             // 需要 #[derive(Debug)]
}

```

工程建议:derive 优先,手写兜底

Debug/Clone/Copy/PartialEq 等"机械性"trait 用 `#[derive(...)]` 自动生成(第10、12章详解);需要定制语义(容差比较、脱敏打印)才手写 `impl`。规则:先 `derive`,不够再手写,永远不要手写机械实现。

易错点 3:运算符重载里误用"消费"还是"借用"

`impl Add for Point` 用 `fn add(self, ...)` 消费两个值。若你的类型很贵且语义是"计算不搬走",应实现 `impl Add for &Point` (对引用加法),或两者都实现。最常见的坑:实现了 `Add for Point` 后, `a + b` 会把 `a` 和 `b` 都 `move` 掉,后续再用 `a` 就编译错误。对策:类型是 `Copy`(如上例 `derive(Clone, Copy)`)就无所谓;非 `Copy` 的昂贵类型,优先实现引用版本,或提供方法而非运算符。

§6.7 trait 继承与默认方法:接口层面的扩展

6.7.1 trait 可以"继承"trait

trait 之间可以声明超集关系:实现子 trait 必须先实现父 trait。这近似 Java 的接口继承,但不带来数据耦合——仍是纯行为契约:

```

trait Shape {
    fn area(&self) -> f64;
}

// Drawable 是 Shape 的超集:实现者必须先实现 Shape
trait Drawable: Shape {
    fn draw(&self) -> String;
}

struct Circle { r: f64 }

impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.r * self.r
    }
}

impl Drawable for Circle {
    fn draw(&self) -> String {
        format!("画圆,半径 {}", self.r)
    }
}

// 泛型约束写法:要求"既是 Shape 又是 Drawable"
fn draw_all<T: Shape + Drawable>(items: &[T]) {
    for it in items {
        println!("{}", it.draw(), it.area());
    }
}

fn main() {
    draw_all(&[Circle { r: 2.0 }]);
}

```

6.7.2 扩展 trait:给别人的类型加方法

trait 最有生产价值的特性: **可以为任何类型实现任何 trait** (孤儿规则:二者至少一个在本 crate)。这让你不修改第三方代码,就能给它的类型挂上自己的行为——C 里想给别人的 struct 加"方法",只能包一层或改源码:

```
// 假设 Point 来自第三方库,我们无权改它

// 自己声明的扩展 trait
trait Distance {
    fn distance_to(&self, other: &Point) -> f64;
}

impl Distance for Point {          // 类型是别人的,trait 是自己的:合法
    fn distance_to(&self, other: &Point) -> f64 {
        let dx = self.x - other.x;
        let dy = self.y - other.y;
        (dx * dx + dy * dy).sqrt()
    }
}

fn main() {
    let p = Point { x: 0.0, y: 0.0 };
    let q = Point { x: 3.0, y: 4.0 };
    println!("距离:{}", p.distance_to(&q));    // 5
}
```

C++/Java	Rust
抽象基类 / 接口	trait(纯行为契约,无数据)
接口默认方法(Java 8+)	trait 默认方法
接口继承	trait 继承(父 trait 约束)
为既有类加接口	扩展 trait(孤儿规则保护全局一致性)
虚函数调用	dyn Trait 的 vtable 查找
模板特化	泛型单态化
装饰器/适配器	组合:包一层 struct 再实现 trait

本章速查表

主题	结论 / 写法
trait 是什么	一组方法签名的契约;不含数据;实现者声明"我能做到"
定义/实现	<code>trait Name { fn m(&self); } / impl Name for Type { ... }</code>
默认方法	trait 内带实现体;实现者可覆盖,不覆盖则用默认
静态分派	泛型 + trait bound: <code>fn f<T: Trait>(x: &T);</code> 编译期单态化,零虚表
impl Trait	参数/返回位置的语法糖:隐藏具体类型,保留 trait 契约
动态分派	<code>Box<dyn Trait> / &dyn Trait;</code> 运行期查 vtable,一次间接调用
胖指针布局	数据指针 + vtable 指针;与 C 手写 vtbl、C++ 虚函数同构
对象安全	方法无泛型参数、不返回 Self;违反则不能 dyn(错误 E0038)
继承	无类继承;行为复用走 trait,数据复用走字段组合
组合	<code>struct Car { engine: Engine } —— has-a 替代 is-a</code>
运算符重载	<code>std::ops::Add</code> 等;注意 self 消费与 <code>&self</code> 两种形态
Display / Debug	<code>{}</code> 与 <code>{:?}</code> ;Debug 常 derive,Display 常手写(定制)
PartialEq	实现 <code>==</code> ;定制规则(容差)手写,机械比较用 derive
trait 继承	<code>trait Sub: Super { ... };</code> 要求先实现父 trait
扩展 trait	类型与他 trait 至少一方是本 crate(孤儿规则);第三方类型可加方法

最小必会清单

- 能说清 trait 与 struct 的分工:数据进 struct,行为进 trait
- 能徒手写出 trait 定义、默认方法、impl 实现与调用
- 能解释静态分派与动态分派的机制差异(展开 vs 查表)与开销差异
- 能在"异构集合"场景正确使用 `Box<dyn Trait>`,并能解释胖指针布局
- 能说出对象安全的两条主要规则,并解释为什么泛型方法不能进 vtable
- 能说出"无继承"的设计:组合(字段)+ trait(契约)分别替代什么
- 能实现 `std::ops::Add` 并为自定义类型定制 Display
- 知道 derive 优先原则:Debug/Clone/Copy/PartialEq 先 derive 再手写
- 能写出 trait 继承(父 trait 约束)与扩展 trait(为第三方类型加方法)

自测题

Q 1. trait 与 C 的函数指针表有什么本质区别?

✓ 参考答案

本质区别是"契约由谁维护":C 的表是程序员手写的,与类型无绑定,void* 丢掉类型信息,字段/方法对应关系靠约定;trait 是编译器参与的类型契约——实现时逐方法检查签名,调用时类型安全,表(如需要 vtable)由编译器生成。同一份代码里,C 靠"记得",Rust 靠"类型保证"。

Q 2. 静态分派与动态分派各有什么开销与限制?生产上如何选择?

✓ 参考答案

静态(泛型/impl Trait):编译期单态化,可内联、可特化,零运行时开销;代价是"编译出一份份专用代码"(代码膨胀,一般可控)、调用方必须知道具体类型。动态(dyn):运行期查 vtable,一次间接跳转,无法内联;换来异构集合与运行期才确定的类型。选择:默认静态;只有异构集合、插件化、运行期决定类型时才用 dyn。

Q 3. 为什么 Vec<dyn Shape> 编译不过,而 Vec<Box<dyn Shape>> 可以?

✓ 参考答案

dyn Shape 是"胖指针"(数据指针 + vtable 指针),大小不固定、不可直接拷贝,不能作为 Vec 的元素按值存储;Box<dyn Shape> 把胖指针装箱到堆上,Box 本身是定长指针,可以放入 Vec。同理,Vec<&dyn Shape> 也可行(借用形式的胖指针)。

Q 4. 一个方法带泛型参数,为什么 trait 就不能做成 dyn?

✓ 参考答案

vtable 是固定大小的函数指针数组,每行存一个确定签名的方法。泛型方法实际是"方法族"(每个具体 T 一份),无法填进固定槽位;而且调用时需要知道 T 才能选实现。所以对象安全要求方法无泛型参数。需要"多态 + 泛型行为"时,改用泛型约束(静态)而不是 dyn(动态)。

Q 5. "组合优于继承"在 Rust 里具体怎么落地?

✓ 参考答案

两层落地:① 数据复用:struct Car { engine: Engine }——把要复用的状态作为字段包含,而不是继承基类;② 行为复用:trait 声明契约,各类型独立实现,相互解耦;需要"既是 A 又是 B"时用 trait bound(Shape + Drawable)或组合。没有 is-a 层级,只有 has-a 结构与行为契约。

Q 6. 运算符重载时,"消费 self"与"借用 &self"两种实现有什么后果?

✓ 参考答案

impl Add for Point(消费)调用 a + b 后 a、b 被 move,无法再用;若 Point 实现 Copy 则无感。impl Add for &Point(借用)调用 &a + &b,原值保留,适合昂贵非 Copy 类型。选择依据:值语义小类型(坐标、颜色)常两者都实现;引用语义类型只实现借用版,或用方法替代运算符。

章末实验题:几何图形系统

实验 6:几何图形系统——trait 全流程实战

实验目标:实现一个"几何图形系统",把本章所有知识点串起来:trait 契约、默认方法、静态分派、动态分派、运算符重载、组合、trait 继承。

实验要求:

- 任务 1:定义 Shape trait(area/name 方法)与 Drawable: Shape(增加 draw 方法)(覆盖:trait 定义 §6.2、trait 继承 §6.7)
- 任务 2:实现 Circle、Rect、Triangle 三个图形(覆盖:impl 契约 §6.2)
- 任务 3:写泛型 total_area 与 impl Trait 参数函数各一个,验证静态分派(覆盖:静态分派 §6.3)
- 任务 4:用 Vec<Box<dyn Drawable>> 建异构场景,统一绘制与计算(覆盖:动态分派 §6.4)
- 任务 5:为 Point 实现 Add(向量加)与 Display(覆盖:运算符重载 §6.6)
- 任务 6:Scene 结构体组合 shapes 与 label 字段,实现 Show 行为(覆盖:组合 §6.5)

实验环境:cargo new geometry;完成后 cargo run,输出应包括:各图形面积、异构列表总面积、每个图形的绘制语句、两个 Point 相加结果。

第一步 **一步:**先写 Point 与其 Add/Display,建立"值类型"地基

第二步 **二步:**写 Shape 与三个图形类型,跑通"各算各的面积"

第三步 **三步:**加 Drawable(继承 Shape),给每个图形补 draw

第四步 **四步:**先写泛型函数(静态),再改造成 dyn 版本,对比两者的使用方式差异

第五步 **五步:**组合 Scene 收尾——体会"数据嵌套 + 行为契约"替代继承的完整形态

参考答案要点

```
// ===== 实验 6:几何图形系统 =====
// 覆盖知识点:trait 契约与默认方法($6.2)、trait 继承($6.7)、
//          静态分派与 impl Trait($6.3)、动态分派 dyn($6.4)、
//          组合优于继承($6.5)、运算符重载与 Display($6.6)

use std::fmt;
use std::ops::Add;

// ---- 任务 1:trait 契约与 trait 继承 ----
trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> &'static str;
}

trait Drawable: Shape {                // 父 trait:必须先实现 Shape
    fn draw(&self) -> String;
}

// ---- 任务 2:三个图形实现契约 ----
struct Circle { r: f64 }
struct Rect { w: f64, h: f64 }
struct Triangle { base: f64, height: f64 }

impl Shape for Circle {
    fn area(&self) -> f64 { std::f64::consts::PI * self.r * self.r }
    fn name(&self) -> &'static str { "圆形" }
}

impl Shape for Rect {
    fn area(&self) -> f64 { self.w * self.h }
    fn name(&self) -> &'static str { "矩形" }
}

impl Shape for Triangle {
    fn area(&self) -> f64 { self.base * self.height / 2.0 }
    fn name(&self) -> &'static str { "三角形" }
}

impl Drawable for Circle {
    fn draw(&self) -> String { format!("画圆:半径 {:.1}", self.r) }
}

impl Drawable for Rect {
    fn draw(&self) -> String {
        format!("画矩形:{:.1} x {:.1}", self.w, self.h)
    }
}

impl Drawable for Triangle {
    fn draw(&self) -> String {
        format!("画三角形:底 {:.1} 高 {:.1}", self.base, self.height)
    }
}

// ---- 任务 3:静态分派(两种写法) ----
```

```

fn total_area<T: Shape>(shapes: &[T]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}

fn describe(shape: &impl Drawable) -> String {
    format!("{}", shape.name(), shape.draw())
}

// ---- 任务 5:运算符重载与 Display ----
#[derive(Clone, Copy)]
struct Point { x: f64, y: f64 }

impl Add for Point {
    type Output = Point;
    fn add(self, other: Point) -> Point {
        Point { x: self.x + other.x, y: self.y + other.y }
    }
}

impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({:.1}, {:.1})", self.x, self.y)
    }
}

// ---- 任务 6:组合:场景拥有图形列表(has-a) ----
struct Scene {
    label: String,
    shapes: Vec<Box<dyn Drawable>>, // 异构:动态分派
}

impl Scene {
    fn new(label: &str) -> Scene {
        Scene { label: String::from(label), shapes: Vec::new() }
    }

    fn add(&mut self, s: Box<dyn Drawable>) {
        self.shapes.push(s);
    }

    fn render(&self) {
        println!("=== 场景:{} ===", self.label);
        let mut total = 0.0;
        for s in &self.shapes {
            println!("{}", s.draw());
            total += s.area(); // dyn 上的方法调用
        }
        println!("场景总面积:{:.2}", total);
    }
}

fn main() {
    // 静态分派:同构集合
    let circles = [Circle { r: 1.0 }, Circle { r: 2.0 }];
    println!("两个圆面积和:{}", total_area(&circles));
}

```

```
// 动态分派:异构集合
let mut scene = Scene::new("小树林");
scene.add(Box::new(Circle { r: 2.0 }));
scene.add(Box::new(Rect { w: 3.0, h: 4.0 }));
scene.add(Box::new(Triangle { base: 2.0, height: 3.0 }));
scene.render();

// 运算符重载与 Display
let p1 = Point { x: 1.0, y: 2.0 };
let p2 = Point { x: 3.0, y: 4.0 };
println!("向量和:{}", p1 + p2);
}
```

设计说明:任务 3 与任务 4 是本章的对照核心——`total_area(&[Circle; 2])` 在编译期展开为圆专用代码,而 `Scene` 里的 `draw/area` 走 `vtable` 查找;两者同一套 `trait`,互不干扰。试着把 `fn add(&mut self, s: Box<dyn Drawable>)` 改成 `s: T` (泛型)就会遇到"异构容器装不下"的限制,这一步亲手撞过,就真正理解了两种分派的边界。

下一章预告:第7章 文件与 IO——C 用 `fopen/fread/fwrite` 与 `errno`,Rust 用 `std::fs` 与 `Result`。路径、读写、目录遍历、序列化与标准流,一次性讲清。